SEOS Filesystem Legacy Info

File System

Architecture

Options:

- FileSystemManager managing a global file system supporting multiple FileSystemDrivers
- Storage Manager with different storage pools, some may contain file system

Requirements

- file system needs a persistent storage backend (Flash, Cloud-Storage ....)
- want exclusive access to persistent storage
- applications need a way to store data permanently
- storage is limited, need to guarantee something and enforce quotas
- guard access to file system from outside of SEOS
- guarantee replay-protection need HW backed monotonic counters eventually
- Formal verification of the file system might not be needed initially if we provide a file content encryption layer, consider file names and directories not security relevant and accept data loss or DoS.

Use Cases

- SEOS Key Store
  - store various blobs
  - basically needs one "folder" where it can store various files
  - may use folders to organize keys, but this would just "look nicer"
- SEOS Configuration Service
  - store configuration data and blobs
  - may just need space for one "database file"
  - may use files and folders
- DataOS
  - need a few files that their database library uses to persist things
- User Applications
  - we don't know what they are doing with the file system
  - may even expect a global file system like in linux, where file access can be shared

Lightweight Alternatives

- "virtual safe"
  - convenient way for applications to put arbitrary data into an encrypted blob
  - it's up to the application to find a way to persist the blob
  - Basically a wrapper around AEAD encryption method of content

- Flash Block Manager
  - can allocate blocks and use them
  - Simple to implement, as it's basically a malloc() for flash
  - Might be used as underlying mechanism for a structured file system driver

File System API

- SEOS organizes files in partitions, where each partition holds a file system.
- the file system API is agnostic of the actual file system implementation. Specific file systems may provide additional functions
- unsupported features in the first version of the API
  - there is no explicit file_create(), use file_open(..., O_CREATE) to create files
  - there is no API for locking files and exclusive access
  - listing files is not supported
  - directories are not supported
  - seeking within a file is not supported
  - renaming files is not supported, must use read(), delete(), open() and write()
  - there is no API for defragmentation
  - there is no API for explicitly flushing any caches

Design Decisions

file_open()

```
hFile = file_open(hPartition, name, flags)
```

- functional description
  - open a file, optionally create the file if it does not exist
  - It is implementation dependent if the function created a new file if the file is missing or returns an error
  - Flags
    - O_CREATE: create file if it does not exist
  - hPartition is a handle for a partition, can be obtained from openPartition() or be a system constant for well defined partitions
  - hFile is a handle representing a file, handle must be closed eventually using file_close()
  - hPartition can be closed once hFile has been obtained, hFile remains valid and usable

file_resize()

```
returnCode_t
file_resize(
    file_handle_t  hFile,
    size_t         newSize,
    uint_t         flags
)
```

- functional description
  - resize the file
  - if the new size is smaller, the content is discarded. Zero is a valid file size
  - if the new size is greater, new content is filled with zeros
  - parameter **`flags`** must be zero, it is reserved for future extensions

file_read()

```
file_read(hFile, offset, len, buffer)
```

- functional description
  - read content into buffer
  - content must be within the file content

file_write()

```
file_write(hFile, offset, len, buffer)
```

- functional description
  - write content into file.

    - for a file with fixed size, the content location must not exceed the size
    - for files with dynamic size, function calls file_resize() first so potential gaps are filled with zeros

FAT File System

General

- sources: demo_filesystem_as_component_pm_as_component or demo_filesystem_as_lib_pm_as_lib
- used the FAT library from: http://elm-chan.org/fsw/ff/00index_e.html
    - BSD License, following disclaimer must be placed in each library file:

```
/*----------------------------------------------------------------------------/
/  FatFs - Generic FAT Filesystem Module  Rx.xx                               /
/-----------------------------------------------------------------------------/
/
/ Copyright (C) 20xx, ChaN, all right reserved.
/
/ FatFs module is an open source software. Redistribution and use of FatFs in
/ source and binary forms, with or without modification, are permitted provided
/ that the following condition is met:
/
/ 1. Redistributions of source code must retain the above copyright notice,
/    this condition and the following disclaimer.
/
/ This software is provided by the copyright holder and contributors "AS IS"
/ and any warranties related to this software are DISCLAIMED.
/ The copyright owner or contributors be NOT LIABLE for any damages caused
/ by use of this software.
/----------------------------------------------------------------------------*/
```

FAT Specifications

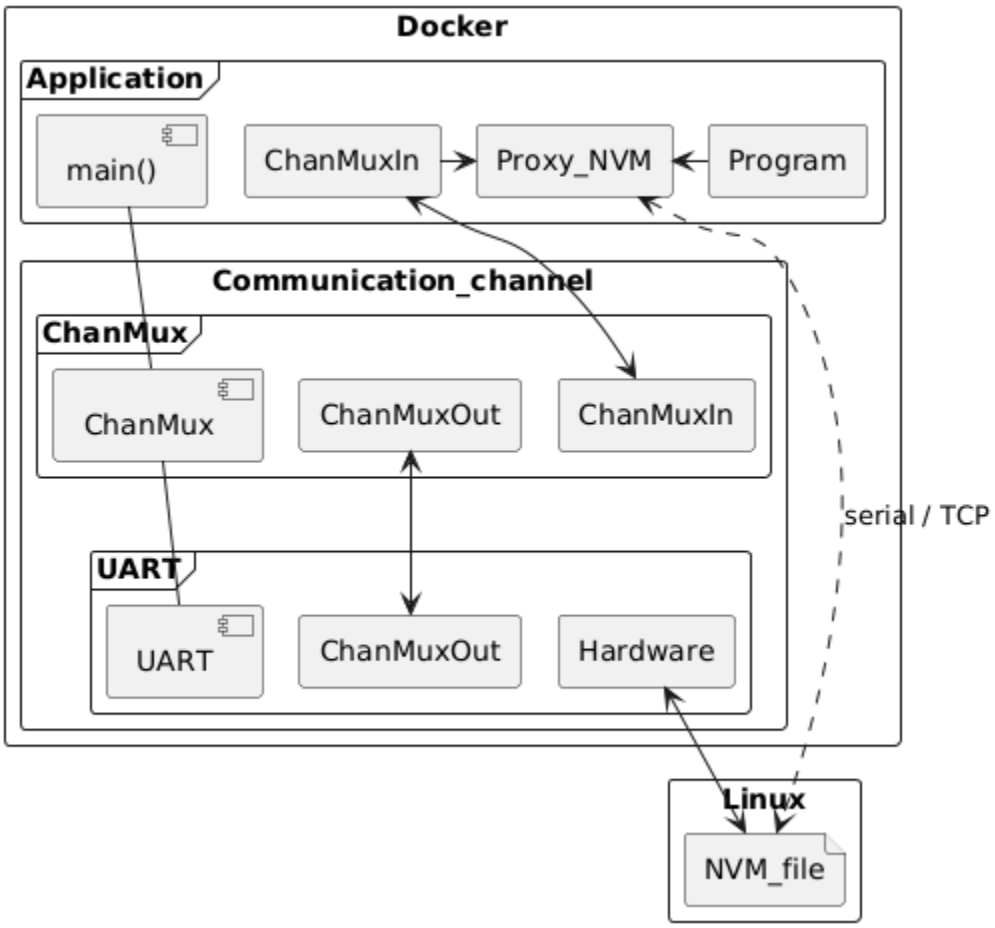
- detailed information about FAT filesystem: https://de.wikipedia.org/wiki/File_Allocation_Table
- FAT format:
    - FAT12
    - FAT16
    - FAT32
    - furthermore there is extFAT, but it's very different from classical FAT formats and thus not supported
- description of the FAT header:
    - https://en.wikipedia.org/wiki/Design_of_the_FAT_file_system
    - https://lowlevel.eu/wiki/File_Allocation_Table
- File / Dir specification
    - 8 + 3 rule
        - 8 char's for file
        - 3 char's for file extension
        - link: https://en.wikipedia.org/wiki/8.3_filename
- Sector
    - smallest sector size: 512 bytes
    - biggest sector size: 4096 bytes
- Cluster
    - A volume's data area is divided up into identically sized clusters, small blocks of contiguous space. Cluster sizes vary depending on the type of FAT file system being used and the size of the partition; typically cluster sizes lie somewhere between 2 KB and 32 KB.
    - smallest cluster size: 1 sector
    - biggest cluster size: depends on sector size
- data will be consistently stored in little-endian format
- Size limits:
    - FAT12
        - 3 sectors for FAT data
        - 1 to 4,084 clusters
        - minimum: 1 sector per cluster × 1 clusters = 512 bytes
        - maximum:
            - 64 sectors per cluster × 4,084 clusters = 133,824,512 bytes
            - 128 sectors per cluster × 4,084 clusters = 267,694,024 bytes
    - FAT16
        - 3 sectors for FAT data
        - 4,085 to 65,524 clusters
        - minimum:
            - 1 sector per cluster × 4,085 clusters = 2,091,520 bytes
            - 1 sector per cluster × 65,525 clusters = 33,548,800 bytes
        - maximum:

        - 64 sectors per cluster × 65,524 clusters = 2,147,090,432 bytes
        - 128 sectors per cluster × 65,524 clusters = 4,294,180,864 bytes
    - FAT32
        - 3 sectors for FAT data
        - 65,525 to 268,435,444 clusters : 512 to 2,097,152
        - maximum:
            - 8 sectors per cluster × 268,435,444 clusters = 1,099,511,578,624 bytes
            - 16 sectors per cluster × 268,173,557 clusters = 2,196,877,778,944 bytes
            - 32 sectors per cluster × 134,152,181 clusters = 2,197,949,333,504 bytes
            - 64 sectors per cluster × 67,092,469 clusters = 2,198,486,024,192 bytes
            - 128 sectors per cluster × 33,550,325 clusters = 2,198,754,099,200 bytes

Setup

Note : In some cases, it will be necessary to store data that should be retained after quitting a SEOS program. So it will be possible to persist data in an external file outside from docker. This requires the use of the “mqtt_proxy_demo” project and the Proxy_NVM object.

Note: The identifier of the file is set internally currently. The name of it is: nvm_06



Next Steps (FS Demo)

- documentation
    - write down detailed build instructions
    - write down what the demo does including architecture pictures
- SEOS updates
    - add all required new error codes
    - update SEOS Runtime Library API requirements
- demo refactoring
    - bootstrapping
        - define a concept how a new system is initialized
        - define a concept how a system boots up and detects if the persistent storage is in a valid state
        - define a concept for what happens if the system ends up in an unrecoverable state
            - simulate errors - how does the file system library actually handle this?
    - extract code to create a partition manager component
        - optional component, can be placed on top of a raw disc driver's I/O API to partition the disk into strictly isolated partitions
        - agnostic of any partition contents and file systems

        - do we have partition headers with some info about content, e.g. file system, access rights .....?
    - basically calculates absolute disk address offset from partition-relative offset

Formal Verification and Cogent ports

- Data61:
    - See also https://ts.data61.csiro.au/projects/TS/cogent.pml
    - All file system implementations (BilbyFs, Ext2, FAT, F2FS) can be deployed in Linux, they sit below Linux's virtual file system switch (VFS) module, using C stub code to provide the top-level entry points expected by the VFS. We are also using them as native components on seL4
    - The Cogent implementations of the file systems share a common library of ADTs that include fixed-length arrays for words and structures, simple iterators for implementing for-loops, and Cogent stubs for accessing a range of Linux APIs such as the buffer cache and its native red-black tree implementation, plus iterators for each ADT.
- Random collection from a google search, to be reviewed:
    - A Verified POSIX-Compliant Flash File System: https://opus.bibliothek.uni-augsburg.de/opus4/frontdoor/index/index/start/2/rows/20/sortfield/score/sortorder/desc/searchtype/simple/query/Ernstflash/docId/3887
    - Formal verification of an extension of a secure, compatible UNIX file system: https://pdfs.semanticscholar.org/a008/830f284ae64f7f742a10768d6b388454c22a.pdf
    - Formal Modeling and Analysis of a Flash Filesystem in Alloy: https://groups.csail.mit.edu/sdg/pubs/2008/abz08_kang_jackson.pdf

File System option for SEOS

BilbyFS

- Cogent port exists
- Input von Gabriele Keller:
    - Der Doktorand begann zuerst, das Filesystem ganz normal in C zu schreiben, allerdings schon mit einem anderen (nicht sehr C kompatiblen) Design
    - Das Cogent Projekt startete erst etwas später, und Sidney schrieb dann sein Filesystem auf Cogent um. Er hat ein paar Operationen in C verifiziert, und später ein paar mit Cogent. Allerdings nicht das vollständige System, weil er dazu keine Zeit mehr hatte. Das ist auch, trotz Cogent, nach wie vor noch ein sehr arbeitsaufwendiger Schritt. Zum Vergleich, dabei handelt es sich, je nach Komplexität des Systems, um etwa gleich viele Zeilen Code wie seL4, dessen Verifikation (ohne die Sackgassen) ca. 10 Personenjahre Aufwand waren.
    - Wir gehen davon aus, nach Sidneys Erfahrung, dass Cogent die Arbeit auf ca. 30% reduziert, also immer noch mindestens 3 Personenjahre.
- Data61 statement:

    - low-level proofs are connected to manual proofs of the correctness of two high-level functions: `sync()` and `iget()`, to demonstrate the ease of verifying the high-level functionality. More complete verification will happen in the future.

Ext2

- too complex for our SEOS needs?
- present Cogent implementation is an almost feature-complete implementation of revision 1. It passes the Posix File System Test Suite, except for the ACL and symlink tests (as we have not implemented those features)

FAT

- simple file system
- License from Microsoft required for commercial usage?
- Cogent port exists, exact state unclear

F2fs

- a native flash file system in Linux
- Cogent port exists, exact state unclear

YAFFS

- https://yaffs.net
- NAND flash file system

SpiFFs

- https://github.com/pellepl/spiffs
- SPI NOR flash file system considerd by Leonard for the Internship
- inspired by YAFFS
- implements wear leveling
- works without a heap, low footprint (16 KiByte of flash and 2 KiByte of RAM to serve 1 MiB SPI flash with 4 files)

- MIT License, see https://github.com/pellepl/spiffs/blob/master/LICENSE

px_log_fs

- https://piconomix.com/fwlib/group___p_x___l_o_g___f_s.html
- record-based file system to store log data on Serial Flash